offline persistence

- a feature that automatically saves and restores the provider's state on the device

Riverpod's offline persistence

- doesn't enforce a specific database or storage method
- riverpod_sqflite package: SQFlite & JSON serialization

Storage

persist

```
final class CustomStorageObject extends Storage<KeyT, EncodedT>
```

```
final class JsonSqFliteStorage extends Storage<String, String>
```

- A storage that stores data in SQLite using JSON.
- Only JSON serializable is supported. This is generally used in combination riverpod_annotation's JsonPersist.

```dart
import 'package:path/path.dart';
import 'package:riverpod_annotation/riverpod_annotation.dart';
import 'package:riverpod_sqflite/riverpod_sqflite.dart';
import 'package:sqflite/sqflite.dart';

part 'storage_provider.g.dart';

@riverpod
FutureOr<JsonSqFliteStorage> storage(Ref ref) async {
  return JsonSqFliteStorage.open(join(await getDatabasesPath(), 'riverpod.db'));
}
```

```
@riverpod
class TodoList extends _$TodoList {
  @override
  FutureOr<List<Todo>> build() async {
    persist(...);

    // ... do something necessary

    return initialValue;
  }
  ...
}
```

```dart
@riverpod
class TodoList extends _$TodoList {
  @override
  FutureOr<List<Todo>> build() async {
    persist(
      ref.watch(storageProvider.future),
      key: 'todos_key’,
      encode: (todos) => jsonEncode(todos.map((e) => e.toJson()).toList()),
      decode: (json) {
        final list = jsonDecode(json) as List<dynamic>;
        return list
            .map((e) => Todo.fromJson(e as Map<String, dynamic>))
            .toList();
      },
      options: const StorageOptions(cacheTime: StorageCacheTime.unsafe_forever),
    );

    await Future.delayed(const Duration(seconds: 3));

    return state.value ?? <Todo>[];
  }
  ...
}
```

```dart
class Todo extends Equatable {
  Todo({
    required this.id,
    required this.description,
    this.completed = false,
    DateTime? updatedAt,
  }) : updatedAt = updatedAt ?? DateTime.now();

  factory Todo.fromJson(Map<String, dynamic> json) {
    return Todo(
      id: json['id'],
      description: json['description'],
      completed: json['completed'],
      updatedAt: DateTime.parse(json['updatedAt']),
    );
  }

  final String id;
  final String description;
  final bool completed;
  final DateTime updatedAt;

  Map<String, dynamic> toJson() => {
    'id': id,
    'description': description,
    'completed': completed,
    'updatedAt': updatedAt.toIso8601String(),
  };

  @override
  List<Object> get props => [id, description, completed, updatedAt];
}
```

```dart
@riverpod
@JsonPersist
class TodoList extends _$TodoList {
  @override
  FutureOr<List<Todo>> build() async {
    persist(
      ref.watch(storageProvider.future),
      options: const StorageOptions(cacheTime: StorageCacheTime.unsafe_forever),
    );

    await Future.delayed(const Duration(seconds: 3));

    return state.value ?? <Todo>[];
  }
  ...
}
```